LAB MANUAL

Advance Web Technology (AWT)

2301CS511

B.Tech. - CSE 6th Semester

A.Y. 2025 – 2026

(Darshan Institute of Engineering and Technology)

Darshan
UNIVERSITY

Table of Content

Lab – 01 Basic JavaScript program.

1.1 Write a program to check that the given number is prime or not.

```
<script>
    n = 23;
    isPrime = true;
    for(i=2;i<=Math.sqrt(n);i++){
      if(n%i==0){
        isPrime = false;
        break;
      }
    }
    if(isPrime){
      document.write("Given Number is a prime number");
    }
    else{
      document.write("Given Number is not a prime number");
    }
  </script>
```

1.2 Write a program to print factors of a given number.

```
</script>
    n = 216;
    for(i=1;i<=n;i++){
      if(n%i==0){
        document.write(i+", ")
      }
    }
</script>
```

1.3 Write a program to implement basic calculator features, take input using prompt function.

```
<script>
    n1 = parseInt(prompt("Enter N1"));
    n2 = parseInt(prompt("Enter N2"));
    op = prompt("Enter Operation");

    ans = 0;
    switch(op){
      case '+':
        ans = n1+n2;
        break;
      case '-':
        ans = n1-n2;
```

```
          break;
        case '/':
          ans = n1/n2;
          break;
        case '*':
          ans = n1*n2;
          break;
      }
      console.log("Ans = ",ans)
    </script>
```

1.4 Write a program to convert celsius to fahrenheit.

```
<script>
    tempInCelsius = prompt("Enter Temprature in Celsius");
    tempInFahrenheit = (tempInCelsius * 9 / 5) + 32
    console.log("Ans = ",tempInFahrenheit)
</script>
```

1.5 Write a program to print given pattern, take number of rows as user input

```
*
* *
* * *
* * * *
```

```
<script>
    n = prompt("Enter Number of Rows");
    for(i=0;i<n;i++){
      for(j=0;j<=i;j++){
        document.write("* ");
      }
      document.write("<br/>");
    }
</script>
```

1.6 Write a JavaScript program to validate user data given from the HTML form

a. username must be of minimum 8 characters
b. password must contain at least 1 digit and 1 special character & should be between 8-12 characters
c. password and repeat password must be same
d. age must be greater than 18 (calculate from date of birth)
e. enrollment must be of 11 digits
f. email validation

```
<form name="myForm" onsubmit="return checkData()">
    <table>
      <tr>
        <td colspan="2" id="errorMsg"></td>
      </tr>
      <tr>
        <td>Enter Username</td>
       <td><input type="text" name="txtUserName" /></td>
      </tr>
      <tr>
        <td>Enter Password</td>
        <td><input type="text" name="txtPassword" /></td>
      </tr>
      <tr>
        <td>Repeat Password</td>
        <td><input type="text" name="txtRePassword" /></td>
      </tr>
      <tr>
        <td>Enter Age</td>
        <td><input type="date" name="txtAge" /></td>
      </tr>
      <tr>
        <td>Enter Enrollment Number</td>
        <td><input type="text" name="txtEnrollmentNumber" /></td>
      </tr>
      <tr>
        <td>Enter Email Address</td>
        <td><input type="text" name="txtEmail" /></td>
      </tr>
      <tr>
        <td colspan="2">
          <button type="submit">Submit</button>
        </td>
      </tr>
    </table>
  </form>
  <script>
    function checkData(){
        myForm = document.forms["myForm"];
        errTd = document.getElementById("errorMsg");

        regExpPass1 = new RegExp("[0-9]");
```

```
        regExpPass2 = new RegExp("[!@#$%^&*()]");
        regExpEnrollment = new RegExp("[0-9]{11}");
        regExpEmail = new RegExp(/^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA
    Z]{2,}$/)

        let dob = new Date(myForm.txtAge.value);
        let currentDate = new Date();
        let age = currentDate.getFullYear() - dob.getFullYear();

        if(myForm.txtUserName.value.length<8){
          errTd.innerHTML = "username must be of minimum 8 characters";
          return false;
        }
        else if(myForm.txtPassword.value.length<8 ||
          myForm.txtPassword.value.length>12 ){
                errTd.innerHTML = "password must contain at least 1 digit and
                1 special character and should be between 8- 12 characters";
                 return false;
        }
        else if(!regExpPass1.test(myForm.txtPassword.value) ||
          !regExpPass2.test(myForm.txtPassword.value) ){
                  errTd.innerHTML = "password must contain at least 1 digit
          and 1 special character and should be between 8- 12 characters";
                      return false;
        }
      else if(myForm.txtPassword.value!=myForm.txtRePassword.value){
            errTd.innerHTML = "password and repeat password must be same";
            return false;
      }
      else if(age<18){
            errTd.innerHTML = "age must be greater than 18";
            return false;
      }
      else if(!regExpEnrollment.test(myForm.txtEnrollmentNumber.value)){
            errTd.innerHTML = "enrollment must be of 11 digits";
            return false;
      }
      else if(!regExpEmail.test(myForm.txtEmail.value)){
            errTd.innerHTML = "invalid email address";
            return false;
      }
    }</script>
```

Lab – 02 Document Object Model Programs

2.1 WAP to change background color on click of button.

```
<button onclick="changeBG()">
    Click
</button>

<script>
    function changeBG(){
      document.body.style.backgroundColor = "red"
    }
</script>
```

2.2 WAP to recognize which mouse event is fired.

```
<button onmousedown="changeBG(event)">
    Click
  </button>

  <script>
    function changeBG(e){
      console.log(e.which)
    }
  </script>
```

2.3 WAP to recognize which keyboard event is fired.

```
<input type="text" onkeydown="changeBG(event)" />

  <script>
    function changeBG(e){
           console.log(e.which)
    }
  </script>
```

2.4 Write a JavaScript that handles following mouse events. Add necessary elements.

a. JavaScript gives the key code for the key pressed.

b. If the key pressed is “a”,”e”,”i”,”o”,”u”, the script should announce that vowel is pressed.

c. When the key is released background should change to blue

```
<input type="text" onkeydown="checkVowel(event)" onkeyup="changeBG()" />
    <script>
        function checkVowel(e){
               console.log(e.key)
               if(['a','e','i','o','u'].includes(e.key)){
                      console.log("Vowel is pressed")
```

```
            }
        }
        function changeBG(){
           document.body.style.backgroundColor = "blue"
        }
    </script>
```

Lab – 03 ES6 Programs

3.1 WAP to demonstrate callbacks in JavaScript.

```
<script>
   function printResult(ans){
     console.log("Result = ",ans);
   }
   function calculate(n1,n2,next){
     ans = n1 + n2;
     next(ans);
   }
   calculate(5,11,printResult)
 </script>
```

3.2 Demonstrate the difference between let and var.

```
<script>
    var a = 5;
    if(true){
      let a = 10;
      console.log("A inside = ",a);
    }
    console.log("A outside = ",a);
</script>
```

3.3 Demonstrate the default parameter while using a function.

```
<script>
    function testMe(a,b=5){
      ans = a + b;
      console.log("ans = ",ans);
    }
    testMe(5,7);
    testMe(5)
  </script>
```

3.4 Demonstrate the spread operator.

```
<script>
contactDetails = {mobile:'123456789', email:'asdf@asdf.com',
city:'rajkot'};
personalDetail = {name:'arjun',age:5}
allDetails = {...contactDetails, ...personalDetail}
</script>
```

3.5 Demonstrate the for of loop.

```
<script>
	a = [1,2,3,4,5]
	for(i of a){
		console.log(i)
	}
</script>
```

Lab – 04 NextJS application setup and project sturecture

4.1 Setup a NextJS project using "npx create-next-app@latest" command.

- System requirements
  Node.js 20.9 or later
- Command to create app:
  npx create-next-app@latest

```
What is your project named? my-app
Would you like to use the recommended Next.js defaults?
    Yes, use recommended defaults - TypeScript, ESLint, Tailwind CSS, App Router, Tu
    No, reuse previous settings
    No, customize settings - Choose your own preferences
```

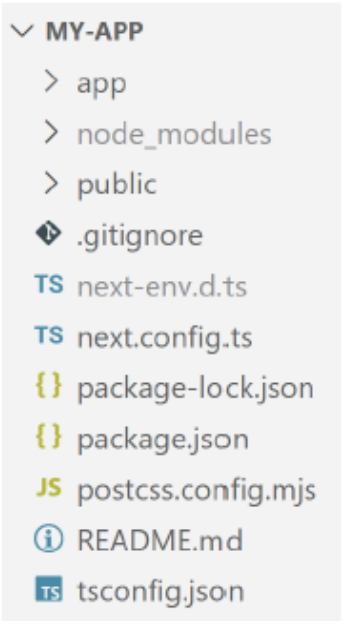
- If you choose to customize settings, you'll see the following prompts:

```
Would you like to use TypeScript? No / Yes
Which linter would you like to use? ESLint / Biome / None
Would you like to use React Compiler? No / Yes
Would you like to use Tailwind CSS? No / Yes
Would you like your code inside a `src/` directory? No / Yes
Would you like to use App Router? (recommended) No / Yes
Would you like to customize the import alias (`@/*` by default)? No / Yes
What import alias would you like configured? @/*
```

- After completing the process folder with the project name will be created and all the dependencies will be installed.
- cd my-app
- Run npm run dev to start the development server.
- Visit http://localhost:3000 to view your application

4.2 Explore Default Project Structure for NextJS created with create-next-app script.

Project structure



top level folders

- **app**: contains App Router
- **public**: contains static assets to be served
- **node_modules**: contains dependencies
- **src** (optional, generated if selected yes in installation process): application source folder

top level files

- **next.config.js**: configuration file for Next.js
- **tsconfig.json**: configuration fole for TypeScript
- **next-env.d.ts**: TypeScript declaration file (do not change)
- **package.json**: manifest file that describe project

A typical Next.js project contains important root-level folders and files such as:

• src/: Main source code folder, houses most application logic.

• app/: Contains nested route structure using App Router; each folder represents a URL segment. Routing logic, page components, layouts, error boundaries, and suspense boundaries reside here.

• components/: Reusable UI elements (buttons, cards, navigation bars).

• lib/: Shared utility functions, helpers, or global modules.

• utils/: Common helpers for tasks such as data formatting and validation.

• styles/: CSS or styling files.

• public/: Static files accessible via root URL (images, fonts, icons).

• next.config.js: Next.js configuration.

• package.json: Project dependencies and scripts.

• node_modules/: Installed packages.

App Directory

• app/ uses a folder-based routing system. Every folder is a segment in the URL; adding page.js or route.js makes a segment publicly accessible.

• Special files in route segments include:

• layout.js: Global or per-route layouts (e.g., shared headers/footers).

• template.js: Allows rendering a unique instance on each navigation.

• error.js: React error boundary file for that segment.

• loading.js: Suspense/loading boundary.

• not-found.js: Custom 404 for that segment.

• Colocate logic and assets relevant to a route inside its segment folder.

Recommended Additional Folders

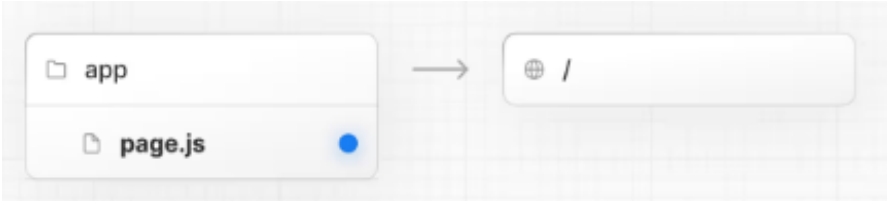
• assets/: Static assets (images, icons, videos) for the UI.

• hooks/: Custom React hooks for shared logic.
• services/: Logic for interacting with APIs, authentication, or external services.
• context/: Context providers for state management (if applicable).

File Naming Conventions and Route Groups

• Use consistent file/folder naming for clarity, especially for shared assets/components.
• Organize static informational pages in route groups (e.g., (info)/about, (info)/terms) for grouping routes without affecting URLs.
• Prefix folders/files with an underscore (_) to indicate privacy (for internals not routed by Next.js).

4.3 Update first page.tsx file to display "Hello World from Next Application"



app/page.tsx

```tsx
export default function Home() {
  return (
    <h1>Hello world from Next.js</h1>
  );
}
```

Lab – 05 Basic Routing and Navigation in NextJS application

5.1 Create a NextJS application with home, contact and about page.

5.2 Create a NextJS application with basic Layout.

app/layout.tsx

```
import Link from "next/link";
export default function RootLayout({
  children,
}: Readonly<{
  children: React.ReactNode;
}>) {
  return (
    <html lang="en">
      <head>
        <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.cs
s"
rel="stylesheet" integrity="sha384
sRIl4kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqkuIAvNWLzeN8tE5YBujZqJLB"
crossOrigin="anonymous" />

      </head>
      <body>
        <div className="container">
          <div className="row">
            <div className="col">
                <nav className="navbar navbar-expand-lg bg-body-tertiary">
                  <div className="container-fluid">
                    <Link className="navbar-brand" href="/">Navbar</Link>
                    <button className="navbar-toggler" type="button" data-bs
toggle="collapse" data-bs-target="#navbarSupportedContent" aria
controls="navbarSupportedContent" aria-expanded="false" aria-label="Toggle
navigation">
                      <span className="navbar-toggler-icon"></span>
                    </button>
                    <div className="collapse navbar-collapse"
id="navbarSupportedContent">
                      <ul className="navbar-nav me-auto mb-2 mb-lg-0">
                        <li className="nav-item">
                          <Link className="nav-link" aria-current="page"
href="/"> Home</Link>
                        </li>
```

```
                        <li className="nav-item">
                          <Link  className="nav-link"  href="/about">  About
</Link>
                        </li>
                        <li className="nav-item">
                          <Link    className="nav-link"    href="/contact">
Contact </Link>
                        </li>
                      </ul>
                    </div>
                  </div>
                </nav>
              </div>
            </div>
            <div className="row mt-2">
                {children}
            </div>
      </div>
      </body>
    </html>
  );
}
```

app/page.tsx

```
export default function Home() {
  return (
    <div className="col">
      <h1>Home page here</h1>
    </div>
  );
}
```

app/about/page.tsx

```
import React from 'react'
function About() {
  return (
    <h1>About Us Page</h1>
  )
}
export default About
```

app/contact/page.tsx

```
import React from 'react'
function Contact() {
  return (
    <h1>Contact Us Page</h1>
  )
}
export default Contact
```

Lab – 06 Apply CSS on NextJS application

6.1 Apply CSS on the static website created in the previous lab.

app/globals.css

```
body {
  background-color: #f8f9fa;
  font-family: Arial, sans-serif;
}

h1 {
  color: #0d6efd;
  text-align: center;
}

.navbar {
  margin-bottom: 20px;
}
```

app/layout.tsx

```
import "./globals.css";
import Link from "next/link";
export default function RootLayout({
  children,
}: Readonly<{
  children: React.ReactNode;
}>) {
  return (
    <html lang="en">
      <head>
        <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.cs
s"
rel="stylesheet" integrity="sha384
sRIl4kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqkuIAvNWLzeN8tE5YBujZqJLB"
crossOrigin="anonymous" />

      </head>
      <body>
        <div className="container">
          <div className="row">
            <div className="col">
```

```
                <nav className="navbar navbar-expand-lg bg-body-tertiary">
                  <div className="container-fluid">
                    <Link className="navbar-brand" href="/">Navbar</Link>
                    <button className="navbar-toggler" type="button" data-bs
toggle="collapse" data-bs-target="#navbarSupportedContent" aria
controls="navbarSupportedContent" aria-expanded="false" aria-label="Toggle
navigation">
                      <span className="navbar-toggler-icon"></span>
                    </button>
                    <div className="collapse navbar-collapse"
id="navbarSupportedContent">
                      <ul className="navbar-nav me-auto mb-2 mb-lg-0">
                        <li className="nav-item">
                          <Link className="nav-link" aria-current="page"
href="/"> Home</Link>
                        </li>
                        <li className="nav-item">
                          <Link  className="nav-link"  href="/about">  About
</Link>
                        </li>
                        <li className="nav-item">
                          <Link    className="nav-link"    href="/contact">
Contact </Link>
                        </li>
                      </ul>
                    </div>
                  </div>
                </nav>
            </div>
          </div>
          <div className="row mt-2">
              {children}
          </div>
      </div>
      </body>
    </html>
  );
}
```

app/about/about.module.css

```
      .title {
        color: green;
```

```
  text-align: center;
  margin-top: 20px;
}
```

about/page.tsx

```
import styles from "./about.module.css";

export default function About() {
  return <h1 className={styles.title}>About Us Page</h1>;
}
```

app/page.tsx

```
export default function Home() {
  return (
    <div className="col text-center">
      <h1 className="text-primary">Home page here</h1>
      <p className="lead">Welcome to our Next.js website</p>
    </div>
  );
}
```

contact/page.tsx

```
export default function Contact() {
  return (
    <div className="text-center">
      <h1 className="text-success">Contact Us Page</h1>
    </div>
  );
}
```

Lab – 07 Route group and dynamic routes in NextJS

7.1 Write an application which demonstrate Route Group and Dynamic Route.

app/(admin)/layout.tsx

```
import React from 'react'

function AdminLayout({
  children,
}: Readonly<{
  children: React.ReactNode;
}>) {
  return (
    <>
      <h1>Admin Layout</h1>
      {children}
    </>
  )
}
export default AdminLayout
```

app/(admin)/books/page.tsx

```
import React from 'react'

function Books() {
  return (
    <div>Books page here</div>
  )
}

export default Books
```

app/(admin)/books/[id]/page.tsx

```
import React from 'react'

async function BooksDetail({
  params,
}: {
  params: Promise<{ id: string }>
}
) {
  const {id} = await params;
```

```
    return (
      <>
        <div>BooksDetail with id = {id}</div>
      </>
    )
  }

  export default BooksDetail
```

app/(auth)/layout.tsx

```
  import React from 'react'
  function AuthLayout({
    children,
  }: Readonly<{
    children: React.ReactNode;
  }>) {
    return (
      <>
        <h1>Authentication Layout</h1>
        {children}
      </>
    )
  }

  export default AuthLayout
```

app/(auth)/login/page.tsx

```
  import React from 'react'

  function Login() {
    return (
      <div>Login page here</div>
    )
  }

  export default Login
```

Lab – 08 Route Handler

8.1 Create Rest API to perform CRUD operation on array using route handler.

app/api/items/route.ts

```
import { NextResponse } from "next/server";

let items: { id: number; name: string }[] = [
  { id: 1, name: "Apple" },
  { id: 2, name: "Banana" },
];

// GET - fetch all items
export async function GET() {
  return NextResponse.json(items);
}

// POST - add new item
export async function POST(req: Request) {
  const body = await req.json();
  const newItem = {
    id: Date.now(),
    name: body.name,
  };
  items.push(newItem);
  return NextResponse.json(newItem, { status: 201 });
}

// PUT - update item by id
export async function PUT(req: Request) {
  const body = await req.json();
  const { id, name } = body;

  const index = items.findIndex((item) => item.id === id);
  if (index === -1) {
    return NextResponse.json({ error: "Item not found" }, { status: 404 });
  }

  items[index].name = name;
  return NextResponse.json(items[index]);
}
```

```
// DELETE - remove item by id
export async function DELETE(req: Request) {
  const body = await req.json();
  const { id } = body;

  const index = items.findIndex((item) => item.id === id);
  if (index === -1) {
    return NextResponse.json({ error: "Item not found" }, { status: 404 });
  }

  const deletedItem = items.splice(index, 1);
  return NextResponse.json(deletedItem[0]);
}
```

Lab – 09 middleware, configuring NextJS application

9.1 Create a middleware in NextJS to add pageNo to be zero if it does not have any parameter in the request.

middleware.ts

```
import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";

export function middleware(req: NextRequest) {
  const url = req.nextUrl;

  // Check if "pageNo" is missing
  if (!url.searchParams.has("pageNo")) {
    url.searchParams.set("pageNo", "0"); // default to 0

    // Redirect with updated query param
    return NextResponse.redirect(url);
  }

  // Continue if already has pageNo
  return NextResponse.next();
}

// Apply middleware only to specific routes (optional)
export const config = {
  matcher: ["/"], // run on homepage, or add more routes like
"/products/:path*"
};
```

9.2 Create a middleware to check for the token in the request, if it does not have token redirect them to login page.

middleware.ts

```
import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";

export function middleware(req: NextRequest) {
  const token = req.cookies.get("token")?.value;
  const url = req.nextUrl;
```

```
  if (!token && url.pathname !== "/login") {
    url.pathname = "/login";
    return NextResponse.redirect(url);
  }

  return NextResponse.next();
}

// Apply middleware only to specific routes
export const config = {
     matcher: ["/((?!login).*)"], // run on all routes except /login
};
```

Lab – 10. Fetching Data from mockapi

10.1 Write a NextJS application to consume getALL api from mockapi and display the data on the page.

```
export default async function Home() {
  // Replace with your own mockapi endpoint
  const res = await fetch("https://something.mockapi.io/api/v1/users", {
    cache: "no-store", // always fetch fresh data
  });

  if (!res.ok) {
    throw new Error("Failed to fetch data");
  }

  const users: {id: string; name: string; email: string}[] = await res.json();

  return (
    <main className="p-6">
      <h1 className="text-2xl font-bold mb-4">Users List</h1>
      <ul className="space-y-2">
        {users.map((user) => (
          <li key={user.id} className="border p-3 rounded-md shadow-sm">
            <p><strong>ID:</strong> {user.id}</p>
            <p><strong>Name:</strong> {user.name}</p>
            <p><strong>Email:</strong> {user.email}</p>
          </li>
        ))}
      </ul>
    </main>
  );
}
```

10.2 Write a NextJS application to consume getByID api from mockapi and display the data on the page

app/users/[id]/page.tsx

```
type User = {
  id: string;
  name: string;
  email: string;
```

```
};

export default async function UserPage({
  params
}: {
  params: { id: string }
}) {

  const res = await fetch(
    `https://something.mockapi.io/api/v1/users/${params.id}`,
    {
      cache: "no-store", // always fetch fresh data
    }
  );

  if (!res.ok) {
    throw new Error("Failed to fetch user data");
  }

  const user: User = await res.json();

  return (
    <main className="p-6">
      <h1 className="text-2xl font-bold mb-4">User Details</h1>

      <div className="border p-4 rounded-md shadow-sm">
        <p><strong>ID:</strong> {user.id}</p>
        <p><strong>Name:</strong> {user.name}</p>
        <p><strong>Email:</strong> {user.email}</p>
      </div>
    </main>
  );
}
```

Lab – 11. Connect with database from NextJS

11.1 Write an application in NextJS to fetch the data from mysql.

1.Install Xampp server and then start control panel and start services of

a. Apache and Db.

b. Go to phpMyAdmin and create database and table as follows.

2. MySQL Table Structure (Product Example)

```
CREATE TABLE products (
 id INT PRIMARY KEY AUTO_INCREMENT,
 title VARCHAR(100),
 price INT,
 category VARCHAR(50),
 description TEXT
);
```

3. Install MySQL Driver

Command:

```
npm install mysql2 - mysql2 allows NextJS to connect with MySQL
```

4. Database Connection File

File: lib/db.ts

Code:

```
import mysql from "mysql2/promise";
export const db = mysql.createPool({
 host: "localhost",
 user: "root",
 password: "",
 database: "awt_db",
});
```

5. Fetch All Products (GET ALL)

File: app/products/page.tsx

Code:

```
import { db } from "@/lib/db";
import Link from "next/link";
export default async function Products() {
const [rows]: any = await db.query("SELECT * FROM products");
return (
<div>
```

```
<h1>Product List</h1>
<table border={1} cellPadding={10}>
<thead>
      <tr>
            <th>ID</th>
            <th>Title</th>
            <th>Price</th>
            <th>Category</th>
            <th>Details</th>
      </tr>
</thead>
      <tbody>
      {rows.map((p: any) => (
            <tr key={p.id}>
                  <td>{p.id}</td>
                  <td>{p.title}</td>
                  <td>₹{p.price}</td>
                  <td>{p.category}</td>
                  <td><Link href={"/products/"+p.id}>Detail</Link></td>
            </tr>
      ))}
      </tbody>
</table>
</div>
);
}
```

6. Fetch Product by ID (GET BY ID)

File: app/products/[id]/page.tsx

Code:

```
import { db } from "@/lib/db";
export default async function Product({ params }: { params:Promise <{
id:string }>}) {
const {id} = await params;
const [rows]: any = await db.query(
      `SELECT * FROM products WHERE id = ${id}`
);

if (!rows || rows.length === 0) {
      return <h2>Product not found</h2>;
}
```

```
const product = rows[0];
return (
      <div>
            <h1>{product.title}</h1>
            <p><strong>Price:</strong> ₹{product.price}</p>
            <p><strong>Category:</strong> {product.category}</p>
            <p>{product.description}</p>
      </div>
);
}
```

Lab – 12. Connect with database using Prisma ORM

12.1 Fetch data from mysql using Prisma ORM

STEP 1: DATABASE SETUP (MySQL)

Create Database:

```
CREATE DATABASE awt_db;
USE awt_db;
```

Create Table:

```
CREATE TABLE products (
 id INT PRIMARY KEY AUTO_INCREMENT,
 title VARCHAR(100),
 price INT,
 category VARCHAR(50),
 description TEXT
);
```

Insert Sample Data:

```
INSERT INTO products (title, price, category, description) VALUES
('Laptop', 55000, 'Electronics', 'High performance laptop'),
('Mobile', 25000, 'Electronics', 'Android smartphone'),
('Headphones', 3000, 'Accessories', 'Noise cancelling'),
('Chair', 7000, 'Furniture', 'Office chair');
```

STEP 2: CREATE NEXTJS PROJECT

```
npx create-next-app awt-lab12
cd awt-lab12
npm run dev
```

STEP 3: INSTALL PRISMA

```
npm install prisma --save-dev
npm install prisma@5 @prisma/client@5
```

For new prisma

```
 install @prisma/client @prisma/adapter-mariadb dotenv
```

Initialize Prisma:

```
npx prisma init
```

This will create a root folder named prisma and a file named schema.prisma inside it.

STEP 4: CONFIGURE DATABASE CONNECTION

File: .env

```
DATABASE_URL="mysql://root@localhost:3306/awt_db"
```

STEP 5: DEFINE PRISMA MODEL

File: prisma/schema.prisma

```
generator client {
 provider = “prisma-client-js”
}

datasource db {
 provider = "mysql"
 url = env("DATABASE_URL")
}

model Product {
 id Int @id @default(autoincrement())
 title String
 price Int
 category String
 description String
}
```

STEP 6: GENERATE PRISMA CLIENT

```
npx prisma db pull
npx prisma generate
```

STEP 7: CREATE PRISMA CLIENT INSTANCE

File: lib/prisma.ts

```
import { PrismaClient } from "@prisma/client";

const globalForPrisma = global as unknown as {
     prisma: PrismaClient;
};

export const prisma = globalForPrisma.prisma || new PrismaClient();
```

STEP 8: FETCH ALL RECORDS (GET ALL)
File: app/products/page.tsx

```
import { prisma } from "@/lib/prisma";

import Link from "next/link";

export default async function Products() {
      const products = await prisma.product.findMany();

      return (
            <div>
                  <h1>Product List</h1>
                  <table border={1} cellPadding={10}>
                  <thead>
                        <tr>
                              <th>ID</th>
                              <th>Title</th>
                              <th>Price</th>
                              <th>Category</th>
                              <th>Details</th>
                        </tr>
                  </thead>
                  <tbody>
                        {products.map((p:any) => (
                              <tr key={p.id}>
                                    <td>{p.id}</td>
                                    <td>{p.title}</td>
                                    <td>₹{p.price}</td>
                                    <td>{p.category}</td>
                                    <td><Link
                                    href={"/products/"+p.id}>Detail</Link
                                    ></td>
                              </tr>
                        ))}
                  </tbody>
                  </table>
            </div>
      );
}
```

STEP 9: FETCH RECORD USING ID (GET BY ID)

Dynamic Route:

app/products/[id]/page.tsx

Code:

```
import { prisma } from "@/lib/prisma";

export default async function Product({ params }: { params: Promise <{
id:string }>}) {
      const {id} = await params;
      const product = await prisma.product.findUnique({
            where: {
                  id: id,
                },
      });
      if (!product) {
            return <h2>Product not found</h2>;
      }
      return (
            <div>
                  <h1>{product.title}</h1>
                        <p><strong>Price:</strong> ₹{product.price}</p>
                        <p><strong>Category:</strong>
                        {product.category}</p>
                        <p>{product.description}</p>
            </div>
      );
}
```

Lab – 13. Server Actions
Lab - 14 Client Component and delete operation
Lab – 15 Insert and update the records using Prisma ORM.

1. Demonstrate NextJS CRUD.

Step 1: Create database

```
CREATE DATABASE awt_taskdb;
USE awt_taskdb;
CREATE TABLE users (
 UserID INT PRIMARY KEY AUTO_INCREMENT,
 UserName VARCHAR(50),
 Password VARCHAR(50)
);
CREATE TABLE tasks (
 TaskID INT PRIMARY KEY AUTO_INCREMENT,
 TaskTitle VARCHAR(100),
 TaskDescription TEXT,
 IsCompleted BOOLEAN,
 UserID INT,
 FOREIGN KEY (UserID) REFERENCES users(UserID)
);
INSERT INTO users (UserName, Password) VALUES
('Chirag', '1234'),
('Lallu', '5678');
INSERT INTO tasks (TaskTitle, TaskDescription, IsCompleted, UserID)
VALUES
('DB Assignment', 'Complete MySQL lab', false, 1),
('AWT Project', 'Build Prisma app', true, 1),
('Testing', 'Unit testing', false, 2);
```

Step 2: Create Project

```
npx create-next-app awt-lab12
cd awt-lab12
```

Step 3: Install prisma using following command

```
npm i prisma --save-dev
npm i @prisma/client
npm i @prisma/adapter-mariadb
create .env file: DATABASE_URL="mysql://root@localhost:3306/awt_taskdb"
```

Step 4: npx prisma init

```
npx prisma db pull (compare number of modals pulled with your db tabels)
npx prisma generate
```

Step 5: create prisma file (lib/prisma.ts)

```
import { PrismaMariaDb } from "@prisma/adapter-mariadb";
import { PrismaClient } from "./generated/prisma/client";
const adapter = new PrismaMariaDb({
host:"localhost",
port:3306,
user:"root",
password:"",
database:"awt_taskdb",
connectionLimit:5
})
export const prisma = new PrismaClient({adapter});
```

Step 6: create a new folder named users (app/users) and then create page.tsx file

```
import { users } from '@/lib/generated/prisma/browser';
import { prisma } from '@/lib/prisma'
import React from 'react'
async function UserList() {
      const data = await prisma.users.findMany();
      return (
            <div>
                  <ul>
                  {
                        data.map((u:users)=>(
                              <li key={u.UserID}>{u.UserName}</li>
                        ))
                  }
                  </ul>
            </div>
      )
}
export default UserList;
```

Step 7: create a new folder named [id] inside users (app/users/[id]) and then create page.tsx file.

```
import { prisma } from '@/lib/prisma'
import Link from 'next/link';
import React from 'react'
async function DetailUser({params}:{params:Promise<{id:number}>}) {
      const {id} = await params;
```

```
        const       data       =       await       prisma.users.findFirst(
        {where:{UserID:Number(id)}})
    return (
        <div>
            <h1>{data?.UserName}</h1>
            <Link href={"/users"}>Back</Link>
        </div>
    )
    }
    export default DetailUser
```

Step 8: Create a server action (/app/actions/DeleteUserAction.tsx)

```
    "use server"
    import { prisma } from "@/lib/prisma";
    import { revalidatePath } from "next/cache";
    import { redirect } from "next/navigation";
    async function DeleteUserAction(id:number){
        await prisma.users.delete({where:{UserID:id}});
        revalidatePath("/users");
        redirect("/users");
    }
    export {DeleteUserAction};
```

Step 9: Create a deleteui (/app/ui/DeleteButton.tsx)

```
    "use client"
    import React from 'react'
    import { DeleteUserAction } from '../actions/DeleteUserAction'
    function DeleteButton(paramas:any) {
        const {id} = paramas;
        return (
            <button          onClick={()=>{DeleteUserAction(id)}}>Delete
        </button>
        )
    }
    export default DeleteButton
```

Step 10: Add Delete UI in list page (/api/users/page.tsx)

```
    import { users } from '@/lib/generated/prisma/browser';
    import { prisma } from '@/lib/prisma'
    import Link from 'next/link';
    import React from 'react'
    import DeleteButton from '../ui/DeleteButton';
```

```
async function UserList() {
      const data = await prisma.users.findMany();
      return (
            <div>
                  <table>
                        <tbody>
                        {
                        data.map((u:users)=>(
                              <tr key={u.UserID}>
                              <td>{u.UserName}</td>
                              <td><Link href={"/users/"+u.UserID}> Detail
                              </Link> </td>
                              <td> <DeleteButton id={u.UserID}/>
                              </td>
                              </tr>
                        ))
                        }
                        </tbody>
                  </table>
            </div>
      )
}
export default UserList
```

Step 11: Modify the code of (/app/users/page.tsx)

```
import { users } from '@/lib/generated/prisma/browser';
import { prisma } from '@/lib/prisma'
import Link from 'next/link';
import React from 'react'
import DeleteButton from '../ui/DeleteButton';
async function UserList() {
      const data = await prisma.users.findMany();
      return (
            <div>
                  <table>
                        <tbody>
                        {
                        data.map((u:users)=>(
                              <tr key={u.UserID}>
                              <td>{u.UserName}</td>
                              <td><Link  href={"/users/"+u.UserID}>Detail
                              </Link> </td>
```

```
                              <td><DeleteButton id={u.UserID} /></td>
                              </tr>
                        ))
                        }
                        </tbody>
                  </table>
            </div>
      )
}
export default UserList
```

Step 12: Modify the code of (/app/users/[id]/page.tsx)

```
import { prisma } from '@/lib/prisma'
import Link from 'next/link';
import React from 'react'
async function DetailUser({params}:{params:Promise<{id:number}>}) {
      const {id} = await params;
      const data = await prisma.users.findFirst({
            where:{UserID:Number(id)},
                  include:{
                        tasks:true
                  }
            })
      return (
            <div>
                  <h1>{data?.UserName}</h1>
                  <ul>
                        {
                        data?.tasks.map((t:any)=>(
                              <li key={t.TaskID}>{t.TaskTitle}</li>
                        ))
                        }
                  </ul>
                  <Link href={"/users"}>Back</Link>
            </div>
      )
}

export default DetailUser
```

Step 13: Create a server action to insert data in the table as

(/app/actions/AddUserAction.tsx)

```
"use server"
import { prisma } from "@/lib/prisma";
import { revalidatePath } from "next/cache";
import { redirect } from "next/navigation";
async function AddUserAction(formData: FormData){

      const UserName = formData.get("UserName") as string;
      const Password = formData.get("Password") as string;
      const data = {UserName, Password};
      await prisma.users.create({data});
      revalidatePath("/users");
      redirect("/users")
}
export {AddUserAction}
```

Step 14: Create a form (/app/users/add/page.tsx)

```
import { AddUserAction } from '@/app/actions/AddUserAction'
import React from 'react'
function AddUser() {
return (
      <div>
            <form action={AddUserAction}>
                  <input type='text' name="UserName"/>
                  <input type='text' name="Password"/>
                  <input type='submit'/>
            </form>
      </div>
)
}
export default AddUser
```

Lab – 16. Testing in NextJS application

16.1 Write a test case to perform unit testing on NextJS application using Vitest.

__tests__/page.test.tsx

```
import { expect, test } from 'vitest'
import { render, screen } from '@testing-library/react'
import Page from '../app/page'

test('Page', () => {
  render(<Page />)
  expect(screen.getByRole('heading', { level: 1, name: 'Home'
})).toBeDefined()
})
```

16.2 Write a test case to perform unit testing on NextJS application using Jest.

```
import '@testing-library/jest-dom'
import { render, screen } from '@testing-library/react'
import Page from '../app/page'

describe('Page', () => {
  it('renders a heading', () => {
    render(<Page />)

    const heading = screen.getByRole('heading', { level: 1 })

    expect(heading).toBeInTheDocument()
  })
})
```

Lab – 17 - 20. Complete REST API in NestJS

First generate resource using “nest generate resource laptops”

src/laptops/laptops.module.ts

```
import { Module } from '@nestjs/common';
import { LaptopsService } from './laptops.service';
import { LaptopsController } from './laptops.controller';
import { TypeOrmModule } from '@nestjs/typeorm';
import { Laptop } from './entities/laptop.entity';

@Module({
  controllers: [LaptopsController],
  providers: [LaptopsService],
  imports:[TypeOrmModule.forFeature([Laptop])],
})
export class LaptopsModule {}
```

src/laptops/laptops.service.ts

```
import { Injectable } from '@nestjs/common';
import { CreateLaptopDto } from './dto/create-laptop.dto';
import { UpdateLaptopDto } from './dto/update-laptop.dto';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository } from 'typeorm';
import { Laptop } from './entities/laptop.entity';

@Injectable()
export class LaptopsService {

  constructor(
    @InjectRepository(Laptop)
    private laptopRepo: Repository<Laptop>
  ){}

  create(createLaptopDto: CreateLaptopDto) {
    return this.laptopRepo.insert(createLaptopDto);
  }

  findAll() {
    return this.laptopRepo.find();
  }
```

```
  findOne(id: number) {
    return this.laptopRepo.findOne({where:{id}});
  }

  update(id: number, updateLaptopDto: UpdateLaptopDto) {
    return this.laptopRepo.update(id,updateLaptopDto);
  }

  remove(id: number) {
    return this.laptopRepo.delete(id);
  }
}
```

src/laptops/entity/laptop.entity.ts

```
import { Column, Entity, PrimaryGeneratedColumn } from "typeorm";

@Entity()
export class Laptop {
  @PrimaryGeneratedColumn()
  id: number

  @Column()
  LaptopName: string

  @Column()
  LaptopBrand: string

  @Column()
  LaptopPrice: number
}
```

src/laptops/dto/create-laptop.dto.ts

```
export class CreateLaptopDto {
  LaptopName: string;
  LaptopBrand: string;
  LaptopPrice: number
}
```

src/laptops/dto/update-laptop.dto.ts

```
import { PartialType } from '@nestjs/mapped-types';
```

```
import { CreateLaptopDto } from './create-laptop.dto';

export class UpdateLaptopDto extends PartialType(CreateLaptopDto) {}
```

src/laptops/laptops.controller.ts

```
import {Controller, Get, Post, Body, Patch, Param, Delete} from
'@nestjs/common';
import { LaptopsService } from './laptops.service';
import { CreateLaptopDto } from './dto/create-laptop.dto';
import { UpdateLaptopDto } from './dto/update-laptop.dto';

@Controller('laptops')
export class LaptopsController {
  constructor(private readonly laptopsService: LaptopsService) {}

  @Post()
  create(@Body() createLaptopDto: CreateLaptopDto) {
    return this.laptopsService.create(createLaptopDto);
  }

  @Get()
  findAll() {
    return this.laptopsService.findAll();
  }

  @Get(':id')
  findOne(@Param('id') id: string) {
    return this.laptopsService.findOne(+id);
  }

  @Patch(':id')
  update(@Param('id')    id:    string,    @Body()    updateLaptopDto:
  UpdateLaptopDto) {
      return this.laptopsService.update(+id, updateLaptopDto);
  }
  @Delete(':id')
  remove(@Param('id') id: string) {
      return this.laptopsService.remove(+id);
  }
}
```

Lab 21 React Hooks (useState, useEffect, useActionState, useCallback)

21.1 Demonstrate useState, useEffect, useActionState, useCallback React Hooks

useState()

```
import { useState } from "react";
function Counter() {
  const [count, setCount] = useState(0);
  return (
    <button onClick={() => setCount(count + 1)}>
      {count}
    </button>
  );
}
```

useEffect()

```
import { useEffect, useState } from "react";
function UserGreeting({ name }) {
  useEffect(() => {
    document.title = `Hello, ${name}`;
  }, [name]);
  return <div>Hello, {name}</div>;
}
```

useActionState()

```
import { useActionState } from "react";

function increment(prevState, formData) {
  console.log(formData.get("FirstName"));
  return prevState + 1;
}

function StatefulForm() {
  const [state, formAction] = useActionState(increment, 0);
  return (
    <form>
      {state}
      <input type="text" name="FirstName" />
      <button formAction={formAction}>Increment</button>
    </form>
  );
}
```

useCallBack()

```
import { useCallback, useState } from "react";

function Button({ onClick }) {
  return <button onClick={onClick}>Click</button>;
}

function Parent() {
  const [count, setCount] = useState(0);
  const handleClick = useCallback(() => setCount(c => c + 1), []);
  return <Button onClick={handleClick} />;
}
```

Lab – 22 React Hooks (useContext, useDebugValue, useDeferredValue)

22.1 Demonstrate useContext, useDebugValue, useDeferredValue React Hooks

useContext()

```
import { createContext, useContext } from "react";

const ThemeContext = createContext("light");

function DisplayTheme() {
  const theme = useContext(ThemeContext);
  return <span>Current theme: {theme}</span>;
}

function App() {
  return (
    <ThemeContext.Provider value="dark">
      <DisplayTheme />
    </ThemeContext.Provider>
  );
}
```

useDebugValue()

```
function useFetch(apiUrl) {
  … …
  useDebugValue(error)
  … …
}
```

useDeferredValue()

```
import { useDeferredValue, useEffect, useState } from "react";

function SearchStudent(){
  const [data,setData] = useState([]);
  const [searchText,setSearchText] = useState("");
  const deferredSearchText = useDeferredValue(searchText);

useEffect(()=>{
    let formattedData = []
    for(let i =0;i<10000;i++){
           formattedData.push(deferredSearchText)
    }
```

```
        setData(formattedData)
      },[deferredSearchText])

    return(
        <>
          <input type="text" onChange={e=>setSearchText(e.target.value)}/>
          <ul>
                {
                   data.map((d)=>(<li>{d}</li>))
                }
          </ul>
        </>
      );
    }
```

Lab – 23 React Hooks (useImperativeHandle, useLayoutEffect, useMemo, useOptimistic)

23.1 Demonstrate useImperativeHandle, useLayoutEffect, useMemo, useOptimistic React Hooks

useImperativeHandle()

```
import { useRef, useImperativeHandle } from 'react';

function MyInput({ ref, ...props }) {
  const inputRef = useRef(null);
  useImperativeHandle(ref, () => {
    return {
      focus() {  inputRef.current.focus();   }
    };
  }, []);
  return <input {...props} ref={inputRef} />;
}

function Form() {
  const ref = useRef(null);
  function handleClick() {
    ref.current.focus();
  }
  return (
    <form>
      <MyInput placeholder="Enter your name" ref={ref} />
      <button type="button" onClick={handleClick}>
        Edit
      </button>
    </form>
  );
}
```

useLayoutEffect()

```
import { useLayoutEffect, useRef } from "react";

function MeasureDiv() {
  const divRef = useRef();

  useLayoutEffect(() => {
    console.log(divRef.current.getBoundingClientRect());
  }, []);
```

```
    return (
      <>
        <div ref={divRef}>Measure me!</div>
      </>
    );
  }
```

useMemo()

```
  import { useMemo, useState } from "react";

  function FetchFakeData() {
    const [something,setSomething] = useState(0);
    const dataNew = useMemo(() => {
        let sum = 0;
        for(let i=0;i<10000000000;i++){
          sum+=i;
        }
        return sum;
    },[]);
    return (
    <div>
            <button onClick={()=> {setSomething(something+1)}}> Increment
        </button>
      sum = {dataNew} - something = {something}
    </div>
  );
  }
```

useOptimistic()

```
  import React, { useState, useOptimistic, useTransition } from 'react';

  // Simulates a slow network request
  const updateLikesInDb = async (newCount) => {
    await new Promise((resolve) => setTimeout(resolve, 1000));
    if (Math.random() < 0.2) {
      throw new Error('Network error');
    }
    return newCount;
  };

  export default function LikeButton({ initialLikes }) {
    const [confirmedLikes, setConfirmedLikes] = useState(initialLikes);
```

```
  const [isPending, startTransition] = useTransition();

  const [optimisticLikes, addOptimisticLike] = useOptimistic(
    confirmedLikes,
    (currentLikes, optimisticUpdate) => currentLikes + optimisticUpdate
  );

  const handleLike = async () => {
    startTransition(async () => {
      addOptimisticLike(1);

      try {
        const updatedCount = await updateLikesInDb(confirmedLikes + 1);
        setConfirmedLikes(updatedCount);
      } catch (error) {
        console.error('Failed to update like count:', error);
      }
    });
  };

  return (
    <button onClick={handleLike} disabled={isPending}>
      {optimisticLikes} Likes
     {isPending && <span style={{ marginLeft: '8px' }}> (...updating)
     </span>}
    </button>
  );
}
```

Lab – 24 React Hooks (useReducer, useRef, useSyncExternalStore, useTransition, useFormStatus)

24.1 Demonstrate useReducer, useRef, useSyncExternalStore, useTransition, useFormStatus React Hooks

useReducer()

```
import { useReducer } from "react";
function reducer(state, action) {
  switch (action.type) {
    case "increment":
      return { count: state.count + 1 };
    case "decrement":
      return { count: state.count - 1 };
    default:
      return state;
  }
}

function Counter() {
  const [state, dispatch] = useReducer(reducer, { count: 0 });
  return (
    <>
    {state.count}
    <button onClick={() => dispatch({ type: "increment" })}>
      Increment
    </button>
    <button onClick={() => dispatch({ type: "decrement" })}>
      Increment
    </button>
    </>
  );
}
```

useRef()

```
import { useRef } from "react";

function AccessDOM(){
  const ref = useRef();

return(
    <>
```

```
        <input type="text" ref={ref}/>
        <button onClick={()=>{
          console.log(ref.current.value);
        }}>Click</button>
      </>
    );
  }
```

useSyncExternalStore() – TodoStore.js

```
let nextId = 0;
let todos = [{ id: nextId++, text: 'Todo #1' }];
let listeners = [];

export const todosStore = {
  addTodo() {
    todos = [...todos, { id: nextId++, text: 'Todo #' + nextId }]
    emitChange();
},
  subscribe(listener) {
    listeners = [...listeners, listener];
    return () => {
      listeners = listeners.filter(l => l !== listener);
    };
  },
  getSnapshot() {
    return todos;
  }
};

function emitChange() {
  for (let listener of listeners) {
    listener();
  }
}
```

useSyncExternalStore() – TodoApp.js

```
import { useSyncExternalStore } from 'react';
import { todosStore } from './TodoStore';

export default function TodosApp() {
```

```
  const todos = useSyncExternalStore( todosStore.subscribe , todosStore.getSnapshot);

  return (
    <>
      <button onClick={() => todosStore.addTodo()}>Add todo</button>
      <hr />
      <ul>
        {todos.map(todo => (
          <li key={todo.id}>{todo.text}</li>
        ))}
      </ul>
    </>
  );
}
```

useTransition()

```
import { useTransition, useState } from "react";

function TransitionExample() {
  const [isPending, startTransition] = useTransition();
  const [value, setValue] = useState("");
  const [data,setData] = useState([]);

  function handleChange(e) {
    setValue(e.target.value)
    startTransition(()=>{
      let tempData = []
      for(let i=0;i<10000;i++){
        tempData.push(e.target.value);
      }
      setData(tempData)
    })
  }

return (
    <>
      <input onChange={handleChange} value={value} />
      {
        data.map(d=><li>{d}</li>)
      }
    </>
```

```
    );
  }
```

useFormStatus()

```
  import { useFormStatus } from "react-dom";

  function SubmitButton(){
    const { pending } = useFormStatus();
    return(
      <>
        <button disabled={pending}>Submit</button>
      </>
    );
  }

  function AddStudent(){
    return(
      <>
        <form action={async ()=>{
          await fetch("apiUrl");
          // code which may take time
        }}>
          <input type="text" name="arjun"/>
          <SubmitButton/>
        </form>
      </>
    );
  }
```

Lab – 25 Redux

25.1 Implement user authentication with Redux toolkit.

store.js

```
import { configureStore } from "@reduxjs/toolkit";
import authReducer from "./authSlice";

const store = configureStore({
  reducer: {
    auth: authReducer,
  },
});

export default store;
```

authSlice.js

```
import { createSlice } from "@reduxjs/toolkit";

const initialState = {
  user: null,
  token: null,
  isAuthenticated: false,
};

const authSlice = createSlice({
  name: "auth",
  initialState,
  reducers: {
    login: (state, action) => {
      state.user = action.payload.user;
      state.token = action.payload.token;
      state.isAuthenticated = true;
    },
    logout: (state) => {
      state.user = null;
      state.token = null;
      state.isAuthenticated = false;
    },
  },
});
```

```
export const { login, logout } = authSlice.actions;
export default authSlice.reducer;
```

Login.js

```
import React, { useState } from "react";
import { useDispatch } from "react-redux";
import { login } from "./authSlice";
import { useNavigate } from "react-router-dom";

export default function Login() {
  const [username, setUsername] = useState("");
  const dispatch = useDispatch();
  const navigate = useNavigate();

const handleLogin = (e) => {
    e.preventDefault();
    // Dummy authentication
    dispatch(
      login({
        user: { name: username },
        token: "fake-jwt-token",
      })
    );
    navigate("/dashboard");
  };

  return (
    <div style={{ margin: "2rem" }}>
      <h2>Login</h2>
      <form onSubmit={handleLogin}>
        <input
          type="text"
          placeholder="Enter username"
          value={username}
          onChange={(e) => setUsername(e.target.value)}
          required
        />
        <button type="submit">Login</button>
      </form>
    </div>
  );
}
```

ProtectedRoute.js

```
import React from "react";
import { useSelector } from "react-redux";
import { Navigate } from "react-router-dom";

export default function ProtectedRoute({ children }) {
  const isAuthenticated = useSelector((state) =>
state.auth.isAuthenticated );

  if (!isAuthenticated) {
    return <Navigate to="/login" replace />;
  }
  return children;
}
```

Dashboard.js

```
import React from "react";
import { useDispatch, useSelector } from "react-redux";
import { logout } from "./authSlice";

export default function Dashboard() {
  const dispatch = useDispatch();
  const user = useSelector((state) => state.auth.user);

  return (
    <div style={{ margin: "2rem" }}>
      <h2>Welcome, {user?.name} </h2>
  <button onClick={() => dispatch(logout())}>Logout</button>
    </div>
  );
}
```

App.js

```
import React from "react";
import { Provider } from "react-redux";
import store from "./store";
import { BrowserRouter as Router, Routes, Route } from "react-router-
dom";
import Login from "./Login";
import Dashboard from "./Dashboard";
import ProtectedRoute from "./ProtectedRoute";
```

```
export default function App() {
  return (
    <Provider store={store}>
      <Router>
        <Routes>
          <Route path="/login" element={<Login />} />
          <Route
            path="/dashboard"
            element={
              <ProtectedRoute>
                <Dashboard />
              </ProtectedRoute>
            }
          />
          <Route path="*" element={<Login />} />
        </Routes>
      </Router>
    </Provider>
  );
}
```

Lab – 27 Chat Application using Socket.io

27.1 Implement chat application using socket.io.

server/index.js

```
import { Server } from "socket.io";
import express from 'express';

const app = express();

const httpServer = app.listen(2000,()=>{ console.log("Server started @
2000")});

const io = new Server(httpServer,{
  cors:{
    origin:"*"
  }
});

io.on('connect',(socket)=>{
  console.log("Someone connected with id = ",socket.id);

  socket.emit('welcome',"welcome to our site");

  socket.broadcast.emit('newUser',`We have a new user with socketID
=${socket.id}`);

  socket.on('join-group',(data)=>{
    socket.join(data);
  })

  socket.on('thanksForWelcome',(data)=>{
    console.log("Msg from the client = ",data)
  })

  socket.on("msgSent", ({input,senderID})=>{
    io.to(senderID).emit("msgRecv",input);
  })

});
```

client/src/ChatApp.js

```
import React, { useEffect, useMemo, useState } from 'react'
import { io } from "socket.io-client";

function ChatApp() {
  const socket = useMemo(()=>io("http://localhost:2000"),[]);

  const [msgs, setMsgs] = useState([]);
  const [input, setInput] = useState("");
  const [senderID, setSenderID] = useState("")

  useEffect(()=>{
    socket.on('welcome',(data)=>{
      console.log("Data from the server = ", data);
      socket.emit('thanksForWelcome',"Thank you for letting me in the
server to chat")
});
    socket.on("msgRecv",data=>{
      setMsgs(oldMsgs=>[...oldMsgs,data]);
    })

    socket.on('newUser',(data)=>{
      console.log(data)
    })
  },[])

  return (
    <div>
      <h6>My Socket ID = {socket.id}</h6>
      <input type='text' value={senderID}
      onChange={e=>setSenderID(e.target.value)} />

      <input type='text' value={input} onChange= { e =>
      setInput(e.target.value)} />
      <button onClick={()=>{socket.emit( " msgSent",{input,senderID})
      }}>Send</button>
      <ul> {msgs.map((m)=>( <li>{m}</li>))} </ul>
    </div>
  )
}
export default ChatApp;
```